Lab 1 · Modelado OLTP → OLAP — PayTrack Colombia

SI7006 Almacenamiento y Procesamiento de Grandes Datos · Maestría en Ciencias de Datos, EAFIT · Prof. Andrés Sacre Alzate
Integrantes: Andres Vélez Rendón · Santiago Vélez Blandón · 23 de octubre de 2026

Entorno y método. PostgreSQL 16.15 en Docker Desktop (Windows), `shared_buffers` = 128 MB. Datos de `generate_data.py`: 500 clientes, 8 comercios, 5 métodos de pago y transacciones repartidas en 90 días; se midió con **50K y 500K** transacciones. Antes de medir se corrió `VACUUM ANALYZE` sobre `oltp.*` y `dw.*`. Todas las lecturas salieron de caché (`shared hit`), así que los tiempos son de CPU. El host introdujo ruido de hasta 2–3×, por eso se reportan **medianas**; la comparación OLTP vs OLAP usa 11 corridas intercaladas por variante.

1. Diagnóstico del OLTP (Parte A)

Consulta	Plan dominante con 50K	Con 500K	Mediana 50K → 500K
Q1 · monto por categoría, 30 días	Bitmap Heap Scan con `idx_txn_created_at` (16 949 filas) + Hash Join a `merchants` + HashAggregate	Cambia a **Parallel Seq Scan** (169K filas)	13,4 → 101,2 ms (×7,6)
Q2 · fraude por segmento	**Seq Scan** de las 50 000 filas + Hash Join a `customers` por `VARCHAR`	Parallel Seq Scan + Hash Join	22,5 → 122,0 ms (×5,4)
Q3 · día × hora, 7 días	Bitmap Heap Scan (4 098 filas) + **Sort por `EXTRACT(DOW)`, `EXTRACT(HOUR)`** + GroupAggregate	Igual, con un sort de 41 601 filas (3,3 MB)	7,2 → 87,6 ms (×12,2)

- **Q1 y Q2 recorren la tabla transaccional completa.** En 3FN el atributo de análisis (`category`, `segment`) vive en otra tabla, así que cada fila requiere un join por una llave `VARCHAR`. Además `transactions` es una tabla pensada para escribir: guarda `status` como texto y llaves alfanuméricas.
- **Q3 calcula `EXTRACT` fila por fila.** El día de la semana y la hora no existen como columnas: no se pueden indexar ni precalcular, y agrupar obliga a ordenar por expresiones.
- **El índice sobre `created_at` casi no ayuda.** Las filas de la ventana están dispersas en disco (correlación física de `created_at` = 0,006), así que el bitmap termina leyendo **todas** las páginas: 569 de 569 con 50K y 5 678 con 500K.

Proyección a 500K transacciones/día durante un año (~182,5 M filas). Al multiplicar los datos por 10, los tiempos crecieron entre 5× y 12× (el paralelismo amortigua Q1 y Q2). Extrapolando de forma lineal según las filas procesadas: **Q2** ≈ 122 ms × 365 ≈ **45 s**, y la tabla pesaría ~16 GB, muy por encima de la caché, así que pasaría de ser CPU a ser I/O; **Q1** (15 M filas en 30 días) ≈ 101 ms × 89 ≈ **9 s**; **Q3** (3,5 M filas en 7 días) ≈ 88 ms × 84 ≈ **7 s**, con un sort de ~280 MB que superaría `work_mem` (4 MB) y se volvería un *external merge* en disco. Son órdenes de magnitud, no mediciones: en un OLTP real las filas llegan en orden cronológico (correlación ≈ 1) y con un año de datos la ventana de 30 días es el 8 % de la tabla, no el 33 %.

2. Decisiones de diseño (Parte B)

Proceso de negocio y grano. El proceso es el pago digital y el grano es **una fila por transacción**. `transaction_id` se conserva como dimensión degenerada y es la PK, así que la base garantiza el grano y rechaza duplicados.

Dimensiones.

- **`dim_date` y `dim_time`** (dadas): traen precalculados el día de la semana, la hora, el fin de semana y el periodo del día, lo que elimina los `EXTRACT`.
- **`dim_customer` (SCD Tipo 2):** `customer_key` (identity), `customer_id` (NK), `city`, `segment`, `signup_date`, `valid_from`, `valid_to`, `is_current`. Un índice único parcial `(customer_id) WHERE is_current` garantiza una sola versión vigente por cliente y `CHECK (valid_from < valid_to)` valida el intervalo. `city` y `segment` son nulables, igual que en el origen, para que Q2 agrupe exactamente lo mismo que el OLTP.
- **`dim_merchant` (SCD Tipo 1):** nombre, categoría y ciudad. La categoría de un comercio cambia muy poco y ninguna pregunta de BI necesita su historia.
- **`dim_payment_method`:** `method_name` (NK) y `method_type` (`card`, `digital_wallet`, `bank_transfer`), un enriquecimiento que no existe en el OLTP. El `CASE` no tiene `ELSE` y la columna tiene `NOT NULL` y `CHECK`: si llega un método sin clasificar, la carga falla en vez de dejar un nulo en silencio.

Lo que no llega al DW. `name`, `email` y `monthly_income_cop` son datos personales y financieros que ninguna pregunta de BI necesita. La Ley 1581 de 2012 (Habeas Data) exige finalidad y minimización; con la natural key se puede volver al OLTP si surge una necesidad legítima y autorizada.

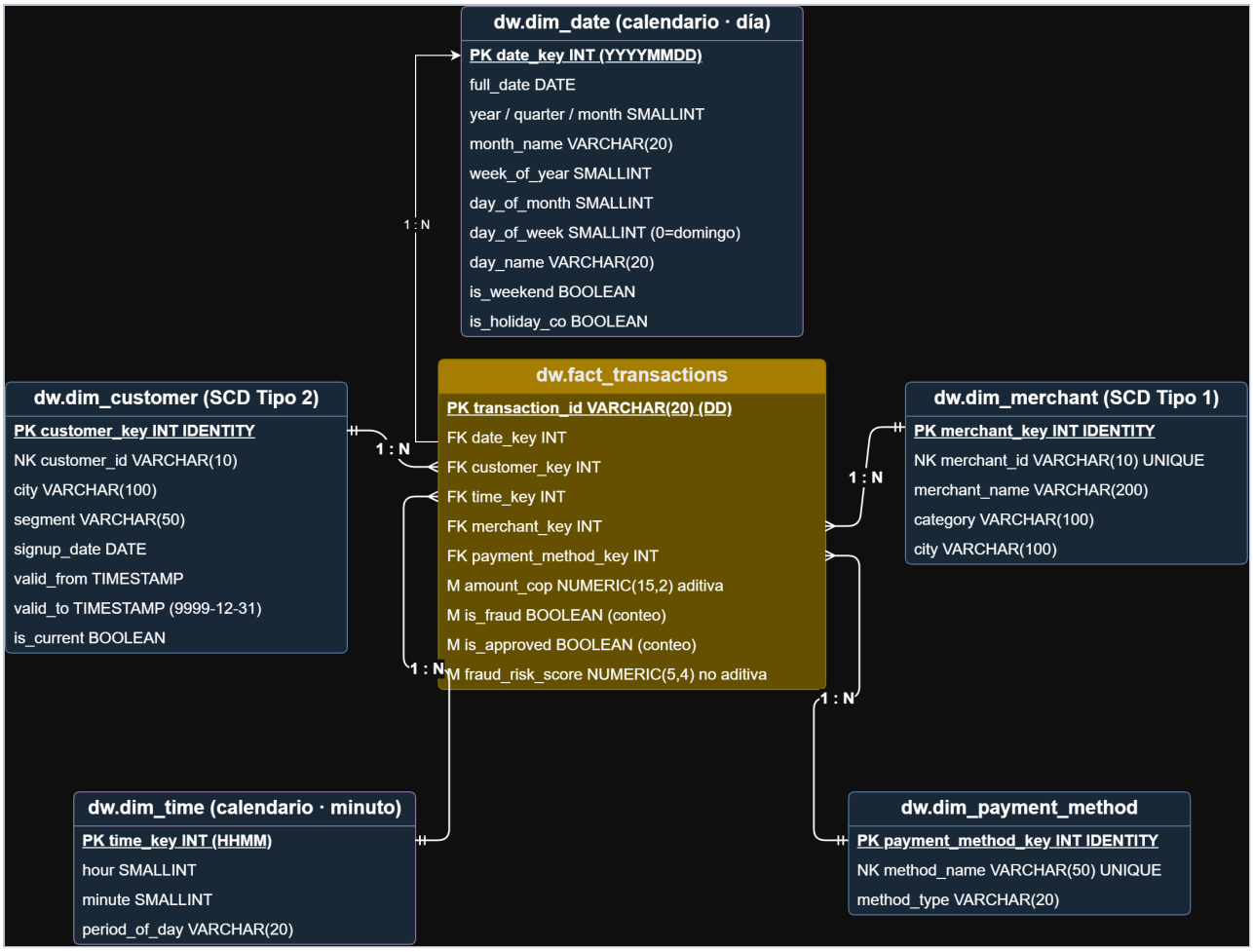


Figura 1. Esquema estrella de PayTrack. Grano: una fila de `fact_transactions` = una transacción. PK = surrogate key, NK = natural key, FK = foreign key, M = medida, DD = dimensión degenerada. Relaciones 1:N de cada dimensión hacia la tabla de hechos.

Medida	Tipo	Aditividad	Cómo se agrega
`amount_cop`	NUMERIC(15,2), exacto	Aditiva	Se suma
`is_fraud`, `is_approved`	BOOLEAN	Aditivas como conteos	`SUM(CASE…)` o `COUNT(*) FILTER`; las tasas se calculan, no se guardan
`fraud_risk_score`	NUMERIC(5,4)	**No aditiva**	Se promedia. Es NULL sin evento de fraude (no 0, para no sesgar el promedio)

Por qué surrogate keys. (1) Permiten varias versiones de un mismo cliente (SCD2), imposible si la PK fuera `customer_id`. (2) Aíslan al DW de cambios o reutilización de las llaves del OLTP y de otras fuentes. (3) Son `INT` de 4 bytes en lugar de `VARCHAR`: la tabla de hechos queda más angosta (49 bytes por fila en promedio, frente a 57 en `transactions`) y los joins son más baratos.

Por qué SCD2 en cliente. `segment` y `city` cambian con el tiempo. Si un cliente pasa de `mass` a `premium` y se sobrescribe (SCD1), todo su fraude y sus ventas históricas quedan atribuidos a `premium` y la tasa de fraude por segmento se distorsiona. Con SCD2 se cierra la versión vieja (`valid_to` = momento del cambio, `is_current = FALSE`) y se abre una nueva. En la carga inicial hay una versión por cliente con `valid_from = created_at` (se verificó que ninguna transacción es anterior al alta) y `valid_to = '9999-12-31'`, que evita NULL en el join por rango. Cada hecho toma la versión vigente **en el momento del pago**: `t.created_at >= valid_from AND t.created_at < valid_to`.

Por qué Q2 no filtra por `is_current`. Cada hecho ya apunta a la versión del cliente vigente cuando pagó, así que el join por `customer_key` atribuye el fraude al segmento que tenía en ese momento. Filtrar por `is_current` eliminaría los hechos ligados a versiones anteriores: las transacciones previas a un cambio de segmento desaparecerían del total.

Carga ELT. `load_star.py` pasa las 9 validaciones y `check_queries.py` da 3/3, con 50K y con 500K. El `fraud_risk_score` se trae con `LEFT JOIN oltp.fraud_events`; se verificó que hay exactamente un evento por transacción fraudulenta (207/207 y 1 942/1 942), así que no se duplican filas. El método de pago se traduce por su natural key (`method_name`), no por el `method_id` serial del OLTP.

3. Rendimiento (Parte D)

Mediana (ms)	50K OLTP	50K OLAP sin índice	50K OLAP final	500K OLTP	500K OLAP sin índice	500K OLAP final
Q1	13,4	19,7	**11,4**	101,2	105,0	106,1
Q2	22,5	20,5	20,5	122,0	122,4	118,5
Q3	7,2	11,2	**2,7**	87,6	67,5	**26,1**

- **Q3 gana 3,4× con 500K.** El plan OLAP hace un Nested Loop sobre las 8 fechas filtradas de `dim_date` + **Index Only Scan** sobre el índice cubriente (`Heap Fetches: 0`) + Hash Join a `dim_time` + HashAggregate sobre columnas `SMALLINT` precalculadas. Desaparecen el `EXTRACT` por fila y el sort por expresiones.
- **Q1 y Q2 empatan con 500K.** Q1 procesa ~169K filas en ambos modelos y su costo es el join y la agregación; Q2 recorre toda la tabla en ambos. La ganancia del join por `INT` frente a `VARCHAR` es pequeña porque las dimensiones son diminutas (500 clientes, 8 comercios).
- **Sin índices, el OLAP puede ser más lento** (Q1 con 50K: 19,7 frente a 13,4 ms). El filtro llega a través de `dim_date.full_date` y el optimizador no lo convierte en un rango sobre `fact.date_key`: hace un Hash Join contra un *seq scan* completo de la tabla de hechos.

Experimento de índices (mediana de 5 corridas por configuración, con 50K y 500K):

- **Índices simples en `date_key`, `time_key`, `customer_key`, `merchant_key`, `payment_method_key` y el compuesto `(date_key, time_key)`: ninguno cambia el plan.** El costo estimado es idéntico al de la tabla sin índices (1 346 / 1 541 / 1 259 con 50K) y todo sigue en *seq scan*. Cada uno ocupa ~3,4 MB con 500K, y los 5 de FK juntos encarecieron la carga de la tabla de hechos un 31 % con 50K (de 6,1 a 8,0 s).
- **Índice cubriente `(date_key) INCLUDE (merchant_key, time_key, amount_cop, is_approved)`:** el único que cambia el plan, a Index Only Scan en Q1 y Q3. Con 50K, Q1 baja de 15,7 a 9,3 ms y Q3 de 10,8 a 2,8 ms; con 500K, Q3 baja de 85 a 30 ms. Su costo es el espacio: ~48 % del heap (2 MB con 50K, 19 MB con 500K).
- **Decisión del TODO 5:** solo el índice cubriente. Requiere el mapa de visibilidad, así que el proceso ELT debe terminar con `VACUUM ANALYZE`.

Por qué el optimizador prefiere el *seq scan*. (1) **Selectividad baja:** Q1 lee el 33 % de las filas y Q2 el 100 %. (2) **Orden físico aleatorio:** la correlación de `date_key` es 0,011–0,015; las filas de un día están en casi todas las páginas, así que un índice no cubriente hace acceso aleatorio a prácticamente toda la tabla y leer en secuencia sale más barato. (3) **Costo del acceso aleatorio:** `random_page_cost = 4` modela una lectura aleatoria como 4 secuenciales; al forzar los índices (`enable_seqscan = off`) el costo estimado se duplica (Q1: 2 684 frente a 1 346) y con 500K los tiempos reales empeoran (Q1: 405 ms, Q2: 561 ms). (4) **Datos pequeños:** con 50K la tabla ocupa 516 páginas (4 MB) y recorrerla cuesta milisegundos. Con `random_page_cost = 1.1` el optimizador sí elige el índice simple en Q3: gana con 50K (4,8 frente a 10,8 ms) pero pierde con 500K (205 frente a 85 ms) por la dispersión de las filas; bajar el parámetro no compensa la falta de orden físico.

Conclusión. En un motor por filas con dimensiones diminutas, el esquema estrella no acelera por sí solo las consultas que recorren toda la tabla (Q1, Q2). Su ventaja está en reemplazar cálculos por fila por atributos precalculados e indexables (Q3: 3,4×), en una tabla de hechos angosta con llaves enteras y en una semántica correcta (historia SCD2, `method_type`, PII excluida). Para llegar a ~180 M filas al año, el siguiente paso es particionar `fact_transactions` por `date_key` (por ejemplo, mensualmente) para que las ventanas lean solo las particiones necesarias, cargarla ordenada por fecha para que sirva un índice BRIN y mantener agregados diarios por categoría y segmento.